

Pickleball Queue & Rating System — Pseudocode + Test Cases

Design Summary

Three separate concerns, three separate data stores:

1. **PlayerRating** (persistent) — Glicko-2 rating, updated only when a match completes.
 2. **ActivePool** (ephemeral) — who's currently in the queue right now. Rebuilt on every join/leave.
 3. **Leaderboard** (derived/read-only) — a view generated from PlayerRating, refreshed periodically, never mutated directly.
-

1. Data Structures

```
Player {
  id
  mu          // rating (default 1500)
  sigma       // rating deviation / uncertainty (default 350)
  volatility   // how erratic results are (default 0.06)
  games_played
  last_active_timestamp
}

ActivePool {
  players: List<Player.id> // just presence, no ranking baked in
}

MatchResult {
  team_a: List<player_id>
  team_b: List<player_id>
  winner: "A" | "B"
  timestamp
}
```

2. Join / Leave (Queue Management)

```
function on_player_join(player_id):
    if player_exists_in_ratings(player_id):
        p = load_rating(player_id)
        inflate_uncertainty_for_inactivity(p)    // Glicko-2 native step
    else:
        p = new Player(
            mu = average_mu(ActivePool) or 1500,    // seed near current pool, not
            sigma = 350,
            volatility = 0.06
        )
        save_rating(p)

    ActivePool.add(player_id)
```

```
function on_player_leave(player_id):
    ActivePool.remove(player_id)
    // NOTHING else happens. No rating change. No leaderboard shuffle.
    // Their persistent rating just sits untouched until they play again.
```

```
function inflate_uncertainty_for_inactivity(p):
    days_away = now() - p.last_active_timestamp
    // Glicko-2's own decay formula, roughly:
    p.sigma = min(350, p.sigma + decay_rate * days_away)
```

Key point: `on_player_leave` is $O(1)$ and touches nothing but the pool. This is what kills the “whole leaderboard messed up” problem — leaving was probably triggering a re-rank pass before.

3. Court / Match Balancing (runs on ActivePool only)

```
function build_next_round(ActivePool, courts_available):
    available = sort(ActivePool, by = mu, descending)
    matches = []

    while available.size >= 4 and courts_available > 0:
        // Greedy: take closest 4 by rating, split into 2v2 minimizing team gap
        group = take_next_4_closest(available)
        team_a, team_b = split_minimizing_gap(group)
        matches.append(Match(team_a, team_b))
```

```

    available.remove(group)
    courts_available -= 1

    leftover = available    // goes to front of next rotation, not penalized
    return matches, leftover

function split_minimizing_gap(four_players):
    // Of the 3 possible 2v2 splits, pick the one with smallest |sum(team_a) - su
    best_split = argmin over [(p0+p1 vs p2+p3), (p0+p2 vs p1+p3), (p0+p3 vs p1+p2
    return best_split

```

This runs fresh every rotation. It never looks at who left last round — only who's here now.

4. Glicko-2 Update (on match completion)

```

function update_ratings(match_result):
    for each player p in match_result.team_a + team_b:
        opponents = the other team's players
        outcomes = [1 if p's team won else 0] * len(opponents)

        g_values = [g(opp.sigma) for opp in opponents]
        e_values = [E(p.mu, opp.mu, opp.sigma) for opp in opponents]

        v = variance(g_values, e_values)
        delta = compute_delta(g_values, e_values, outcomes, v)

        new_volatility = solve_volatility(p.volatility, delta, p.sigma, v) // it
        new_sigma_star = sqrt(p.sigma^2 + new_volatility^2)
        new_sigma = 1 / sqrt(1/new_sigma_star^2 + 1/v)
        new_mu = p.mu + new_sigma^2 * sum(g_values[i] * (outcomes[i] - e_values[i]

        p.mu = new_mu
        p.sigma = new_sigma
        p.volatility = new_volatility
        p.games_played += 1
        p.last_active_timestamp = now()

    save_rating(p)

// helper: g() reduces impact of matches vs high-uncertainty opponents
function g(sigma):
    return 1 / sqrt(1 + 3 * sigma^2 / pi^2)

```

```
function E(mu, opp_mu, opp_sigma):  
    return 1 / (1 + exp(-g(opp_sigma) * (mu - opp_mu)))
```

(Full Glicko-2 volatility solver is the standard published algorithm — Mark Glickman’s paper. Worth implementing exactly as-is rather than approximating; the iterative solve is what keeps volatile players from swinging wildly.)

5. Leaderboard (derived view, not source of truth)

```
function render_leaderboard():  
    return sort(all PlayerRatings, by = conservative_rating, descending)  
  
function conservative_rating(p):  
    return p.mu - 2 * p.sigma    // Dota/TrueSkill-style: "worst case" rank to pre  
                                // one lucky game vaulting an uncertain player
```

Refresh this on a timer (e.g., every 5 min or after each round completes) — never inline it into join/leave.

Test Cases & Rundowns

Test 1 — Player leaves mid-session, no rating impact

Setup: Pool = [A(1600), B(1550), C(1500), D(1480), E(1520)]. Round in progress on 1 court: A+B vs C+D. E is waiting.

Action: D leaves before the match finishes.

Expected:

- `on_player_leave(D)` fires → D removed from ActivePool only.
- The in-progress match is unaffected in this pseudocode’s model (real system should either void or let it finish 3v2 → handle via your existing forfeit rule; rating engine itself doesn’t care).
- D’s `mu=1480`, `sigma=σd` stays frozen in storage.
- Leaderboard unchanged for every other player.

Rundown: Nothing cascades. This is the direct fix for “leaving messes up the whole leaderboard” — previously leaving probably triggered a full re-sort/re-weight pass across

everyone.

Test 2 — New player joins, gets placed fairly without distorting leaderboard

Setup: ActivePool average $\mu = 1540$. New player F joins, never played before.

Action: `on_player_join(F)` .

Expected:

- F seeded at $\mu=1540$ (pool average, not flat 1500), $\sigma=350$.
- F is NOT inserted into the persistent leaderboard at rank based on 1540 with false confidence — $\text{conservative_rating}(F) = 1540 - 700 = 840$, so F shows near the bottom of the leaderboard view until uncertainty shrinks, even though their “true” μ guess is mid-pack.
- F is immediately eligible for court balancing since ActivePool doesn’t care about leaderboard position, only μ for grouping.

Rundown: This resolves the “new player instantly at rank 1 or unfairly stuck at the bottom” problem — display rank is conservative, but matchmaking uses the raw estimate so games stay balanced from game one.

Test 3 — Returning player after 3 weeks away

Setup: G played last 3 weeks ago: $\mu=1650$, $\sigma=60$ (was very confident/stable).

Action: `on_player_join(G)` .

Expected:

- `inflate_uncertainty_for_inactivity(G)` bumps σ upward (e.g., $60 \rightarrow 180$) proportional to days away.
- μ stays 1650 (no reason to assume they got worse, just less certain).
- Next few matches will move G’s μ faster than a “warm” 1650 player would, because higher $\sigma = \text{bigger } \mathbb{E}() / \text{delta sensitivity}$.

Rundown: G doesn’t get frozen at a stale, possibly-wrong rank, but also isn’t punished — the rating adjusts itself through normal play within a few games rather than needing a manual reset.

Test 4 — Uneven pool size, court balancing with leftovers

Setup: ActivePool after joins/leaves = [1600, 1590, 1500, 1490, 1470, 1440, 1300]. 7 players, 1 court (needs 4).

Action: `build_next_round(pool, courts_available=1)` .

Expected:

- Closest 4 by mu selected: {1600, 1590, 1500, 1490} (tightest cluster).
- Split minimizing gap: likely (1600+1490) vs (1590+1500) → gap $|3090-3090| = 0$, best possible.
- Leftover = [1470, 1440, 1300] → carried to next round, prioritized first next rotation (add a `rounds_waited` counter if you want strict fairness — not shown above, easy add).

Rundown: Balancing only ever looks at the *current* pool snapshot. A player leaving before this function runs simply shrinks the input list — no special-case code needed, no leaderboard recompute triggered.

Test 5 — Mid-round leave affecting only pool, not history

Setup: Match completes: Team A (1600,1500) beats Team B (1590,1490). H, previously in Team B, leaves the venue immediately after the loss.

Action:

1. `update_ratings(match_result)` runs first (match already finished — rating update always fires on completion regardless of subsequent queue actions).
2. Then `on_player_leave(H)` fires.

Expected:

- H's mu decreases per the loss (Glicko-2 update already applied) — this **did** happen, correctly.
- H removed from ActivePool.
- No further changes. H's updated (lowered) rating is exactly what should persist for next time they join.

Rundown: Order of operations matters — always resolve match completion and rating updates *before* processing a leave/join event that happens around the same time, so results aren't lost or double-counted. Worth enforcing this as an explicit event-queue ordering rule

in implementation (timestamp-sequenced event log rather than reacting to UI actions instantly).

Why this fixes your specific problem

Your original symptom — “leaderboard and balancing break when people join/leave” — happens when rank position is treated as *state* that must be kept consistent, rather than as a *computed view*. The moment you:

- stop writing to “rank” directly,
- stop letting queue membership affect rating,
- and stop recalculating history on every join/leave,

...the whole class of bugs disappears, because there’s no shared mutable rank state left to corrupt. This is exactly the shift Dota made early on (session-based W/L countable rank → persistent uncertainty-aware MMR decoupled from queue state).